

# Comparing Breadth-First Search (BFS), Depth-First Search (DFS), and A\* Search in Solving the 8-Puzzle Problem

Kimberly M. Bandillo

**Abstract—** The 8-puzzle problem involves artificial intelligence and an algorithm to arrange the 3x3 grid of numbered tiles to reach its goal state. In this study we compare the Breadth-First Search (BFS), Depth-First Search (DFS), and the A\* Search in terms of their performance, execution time, and space occupied while solving the puzzle. Each algorithm will be evaluated based on its path cost and explored states using the provided test cases. The results will provide an insight into their strengths and weaknesses in solving similar type of problems.

## I. INTRODUCTION

To analyze the algorithm, we will test each one using the provided test cases. Then based on the results obtained, we will answer the following question:

*A. Among the three, which is the most appropriate approach for solving the 8-puzzle game? Which is the least appropriate approach?*

The Depth-first search (DFS) algorithm is a good choice for finding solutions to the 8-puzzle game, especially when the target configuration is closer to the leaf nodes of the search tree. DFS explores paths extensively before backtracking, which makes it efficient in finding solutions that are located at shallower levels of the search tree. However, DFS might not be the best option when searching for the shortest path, as it does not guarantee an optimal solution in terms of the fewest steps.

BFS is an excellent option when it comes to searching for the shortest path in the 8-puzzle game. It works by exploring all possible paths in a systematic way, level by level. This ensures that the first solution found is the shortest one, which is perfect when looking to optimize for the fewest moves. However, it's worth noting that BFS can require a lot of memory and may not be practical if the branching factor of the search tree is very high.

A\* Search is often considered the most appropriate approach when it comes to solving problems like 8-puzzle game. The algorithm combines the benefits of both BFS and DFS by taking into account both the cost to reach a particular state and the estimated cost to reach the goal from that state. It is a heuristic search algorithm that can provide optimal solutions while considering the efficiency of exploration.

Presented to the Faculty of the Institute of Computer Science, University of the Philippines Los Baños in partial fulfillment of the requirements for the Degree of Bachelor of Science in Computer Science

A\* Search uses a priority queue to expand states with lower estimated costs first, making it a balanced choice for the 8-puzzle game. Overall, A\* Search is a reliable and efficient algorithm that is suitable for solving a wide range of problems.

*B. What is the difference between informed and uninformed search strategies in terms of the behavior and performance.*

Informed search strategies involve heuristics in their search process. As a result, solutions are found more efficiently and quickly. On the other hand, uninformed search strategies do not utilize any guidance of heuristics and explore all possible paths using their own strategies. This approach allows them to find solutions, but it can be slower and less efficient, particularly for large data. The choice between informed and uninformed search depends on the specific problem, memory cost and efficiency.

## II. METHODS

Using time and psutil libraries in python, the time and memory used by the algorithms are measured. There are two test cases used.

|                  | DFS   | BFS | A* |
|------------------|-------|-----|----|
| Path cost:       | 62331 | 5   | 5  |
| Explored states: | 78213 | 45  | 6  |

Fig. 1. Test Case 1: 1 2 3 4 8 0 7 6 5

|                  | DFS | BFS | A* |
|------------------|-----|-----|----|
| Path cost:       | 6   | 6   | 6  |
| Explored states: | 7   | 90  | 7  |

Fig. 2. Test Case 2: 2 3 0 1 5 6 4 7 8

*1) Table Interpretation:* The two test cases demonstrate different behaviors of three algorithms. In the first case, DFS explores the highest number of states among the three, followed by BFS. In the second case, BFS explores the highest number of states. Both cases show that A\* Search explores the least number of states, indicating that it is the fastest algorithm to use in both cases.

### III. RESULTS AND DISCUSSION

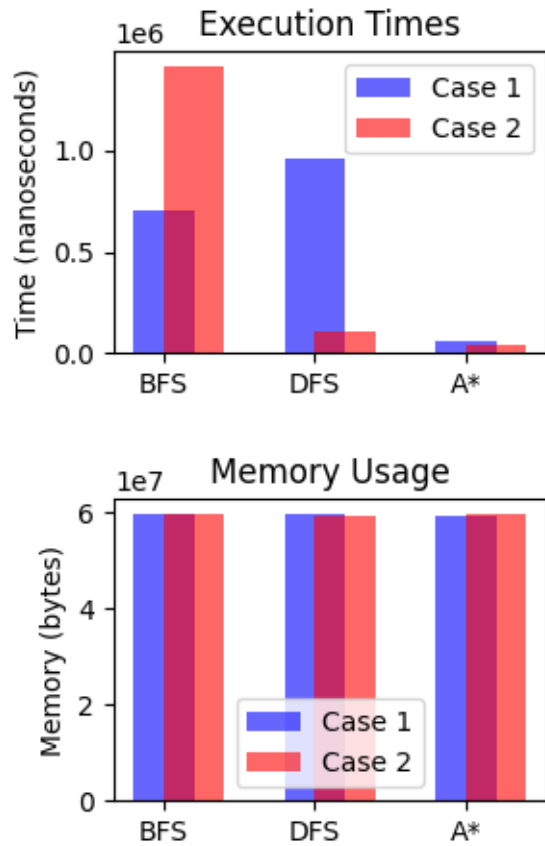


Fig. 3. Comparison of Algorithm Performance: Execution Times and Memory Usage

1) *Graph Interpretation:* According to the graph, A\* Search algorithm has the fastest execution time and is the most consistent among the three algorithms tested. Both BFS and DFS algorithms showed inconsistent performance due to their dependence on the puzzle configuration. Although the memory usage of the three algorithms differed slightly, A\* Search was still the most suitable algorithm to use in this type of problem due to its informed search approach that employs heuristics to make searching more efficient and faster.

### IV. CONCLUSION AND FUTURE WORK

In summary, BFS explores breadth-first, DFS explores depth-first, and A\* Search uses different strategies in searching for the optimal solution. BFS and DFS are uninformed search which has a tendency to have a slower performance compared to informed search like A\* Search.